

67-101
16/1821

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

- 67101 – מבוא למדעי המחשב - מואץ
67130 – מבוא למדעי המחשב - רגיל
67108 – מבוא למדעי המחשב לתלמידי תלפיות

תאריך: 20.1.2010
משך הבחינה: שלוש שעות

מועד א' - סמסטר א' – תשע"א – 2010-2011
פרופ' דני לישצ'ינסקי, פרופ' יובל רבני, מר אלעד מזומן

הנחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיוטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
- פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
- בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.
- בפתרון השאלה הינכם רשאים לבצע שימוש בשיטות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בשיטות משאלות אחרות!)

67.101
16/6"8 PM

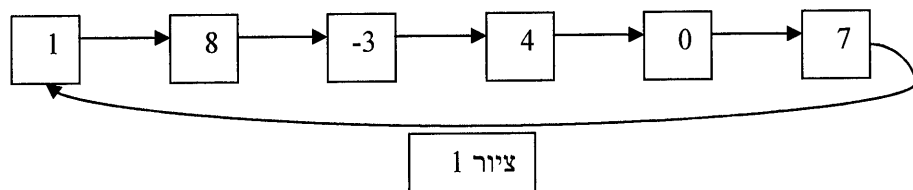
שאלה 1 (33 נקודות)

רשימה מקושרת מעגלית

נתונה המחלקה הבאה לייצוג חוליה ברשימה מקושרת:

```
public class Node {
    public int _data;
    public Node _next;
    public Node(int data, Node next);
}
```

רשימה מקושרת מעגלית הינה רשימה אשר בה שדה ה-`_next` של האיבר האחרון מצביע על האיבר הראשון ברשימה. למשל, הרשימה הבאה היא רשימה מקושרת מעגלית:



לשם הפשטות, הניחו כי ברשימה מקושרת מעגלית כלשהי אין שתי חוליות המכילות את אותו המספר. לכל רשימה מעגלית, נגדיר את אחת מהחוליות כעוגן.

א. (12 נקודות)

עליכם לממש שיטה לא רקורסיבית:

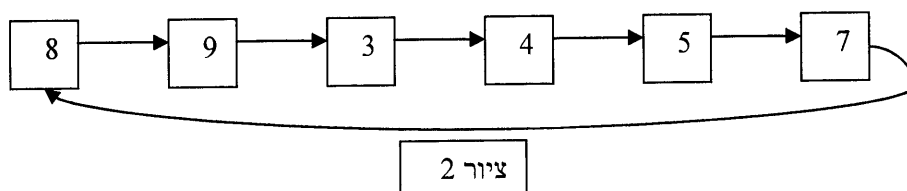
```
public static Node findNumber(Node list1, int x)
```

המקבלת כקלט עוגן `list1` לרשימה מעגלית (המצביע על חוליה כלשהי ברשימה) ומחזירה את המצביע לחוליה שלפני החוליה שמכילה את המספר `x`. אם אין חוליה המכילה את `x` ברשימה, על השיטה להחזיר `null`. במידה ורשימה היא בעלת חוליה אחת בלבד המכילה את `x`, על השיטה להחזיר את המצביע לחוליה זו.

למשל, בהתייחס לרשימה בציור 1 שלעיל, אם ה-`x` הנתון הוא 0, על השיטה להחזיר את המצביע לחוליה שמכילה את 4. אם ה-`x` הנדרש הוא 1, על השיטה להחזיר את המצביע לחוליה שמכילה את 7. אם ה-`x` הנדרש הוא 9, על השיטה להחזיר `null`. וכל זאת, ללא תלות בחוליה שעליה מצביע העוגן `list1`.

ב. (18 נקודות)

רשימה מקושרת מעגלית ממוינת הינה רשימה מקושרת מעגלית אשר בה כל חוליה ברשימה, למעט חוליה אחת בלבד, מצביעה על חוליה המכילה מספר גבוה יותר. למשל, הרשימה הבאה היא רשימה מקושרת מעגלית ממוינת:



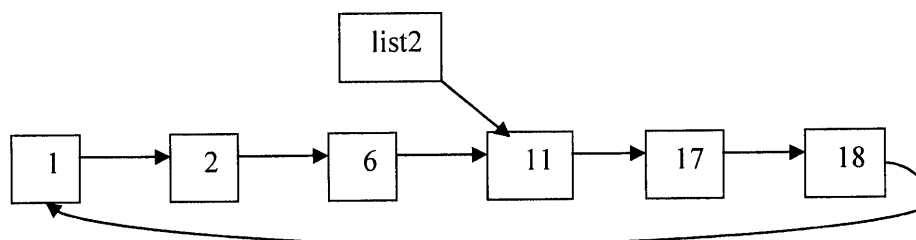
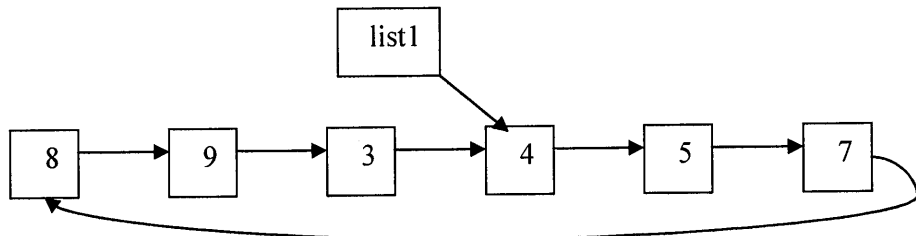
המשך השאלה בעמוד הבא...

67. 101
1/10/16

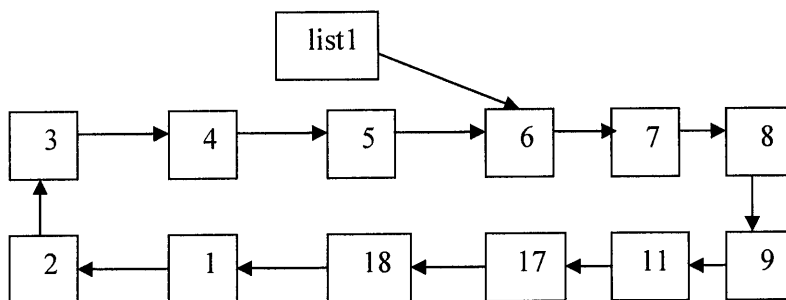
עליכם לממש שיטה לא רקורסיבית:

`public static void merge(Node list1, Node list2)`

המקבלת כקלט שני עוגנים לרשימות מעגליות ממוינות זרות (כלומר, בין שתי הרשימות אין איברים משותפים). על השיטה למזג באופן ממוין את רשימה list2 לתוך רשימה list1. למשל, אם נתונות שתי הרשימות הבאות:



אזי, לאחר הפעלת השיטה merge, הרשימה list1 תתקבל כ:



הערות לסעיף זה בלבד:

1. הנכם רשאים להניח כי בכל רשימה יש לפחות 3 חוליות.
2. אינכם נדרשים לשמור על שלמות הרשימה list2.
3. אינכם רשאים ליצור חוליות חדשות. עליכם לבצע שימוש בחוליות הקיימות של שתי הרשימות בלבד!
4. לאחר הפעלת השיטה, list1 צריך להצביע על חוליה כלשהי ברשימה הממוזגת (אך לאו דווקא על החוליה שעליה הוא הצביע לפני הפעלת השיטה).

ג. (3 נקודות)

בהנחה שמספר האיברים ב-list1 הוא n, ומספר האיברים ב-list2 הוא m, מהי סיבוכיות זמן הריצה של השיטה שמימשתם בסעיף ב' כתלות ב-n ו-m? נמקו!

67. 101
10/10"8P1

שאלה 2 (33 נקודות)

I. קראו בעיון את הקוד הבא:

```
class A {
    static public int val1 = 0;
    public int val2;

    public A(int val) { val1++; val2 = val; }
    void print() {
        for (int i = 0, x = 1; i < val1; i++, x *= val2) {
            System.out.print(x + " ");
        }
        System.out.println();
    }
}

class B extends A {
    public B(int val) { super(val); }
    void print() {
        for (int i = 0, x = 0; i < val2; i++, x += val1) {
            System.out.print(x + " ");
        }
        System.out.println();
    }
}

class Test {
    static public void main(String args[]) {
        1    foo(5);

        2    B b = new B(6);
        3    bar(b);
        4    b.print();
    }

    static private void foo(int n) {
        for (int i = 1; i <= n; i++) {
            A a = new A(i);
            a.print();
        }
    }

    static private void bar(A a) {
        a.print();
        a.val2++;
        a = new A(1);
    }
}
```

המשך השאלה בעמוד הבא...

67.101
16/6" & P11

א. (6 נקודות)

הסבירו מה מבצעת מתודת ה-`print()` של המחלקה A? רשמו את הפלט שיודפס בעת ביצוע הקריאה ל-`foo` בשורה 1 ב-`main`?

ב. (5 נקודות)

מהי סיבוכיות זמן הריצה של המתודה `foo` כתלות בפרמטר `n`?

ג. (6 נקודות)

רשמו את הפלט שיודפס בעת ביצוע שורות הקוד 2,3,4 ב-`main`?

ד. (2 נקודות)

הסבירו מה הם לדעתכם שני הליקויים המשמעותיים ביותר בקוד הנתון (מלבד שימוש בשמות חסרי משמעות והעדר תיעוד)?

II. הקוד הבא מהווה חלק ממימוש של עץ בינארי:

```
class BinaryNode {  
  
    private int      val;  
    private BinaryNode a, b;  
  
    public BinaryNode(int val) {  
        this.val = val;  
        a = b = null;  
    }  
  
    public boolean whatDoIDo() {  
  
        boolean result = true;  
        int v = val;  
        if (a != null) {  
            result = (v <= a.val) && a.whatDoIDo();  
            v = a.val;  
        }  
        return  
            result && (b==null || (v<=b.val) && b.whatDoIDo());  
    }  
}
```

ה. (6 נקודות)

מה הפעולה המבוצעת ע"י המתודה `whatDoIDo()`?

ו. (4 נקודות)

תנו שתי דוגמאות לא טריויאליות (כאשר העץ מכיל לפחות 5 פריטים) שעבורן המתודה תחזיר שני ערכים שונים.

ז. (4 נקודות)

מה ניתן לומר על זמן הריצה של המתודה הנ"ל?

שאלה 3 (33 נקודות)

רקורסיה

כחלק ממערכת של רשת חברתית לא-סימטרית מוגדרת המחלקה Person, שמתארת אדם ברשת. נאמר שברשת יש n אנשים. לכל אדם יש מערך של חברים ברשת, כשהגדרה של חבר כאן אינה סימטרית: יכול להיות שאריק מופיע במערך החברים של בנץ אבל בנץ לא מופיע במערך החברים של אריק. להלן חלק מהגדרת המחלקה:

```
public class Person {
    private String _name;
    private Person[] _friends;
    ...
}
```

כחלק מהרשת אתם נדרשים לספק את השירות של מציאת "מסלול חברים" מאדם לאדם דרך הרשת. למשל אם אריק מופיע במערך החברים של בנץ, ועוגי מופיע במערך החברים של אריק, אז קיים מסלול חברים באורך 3 (שמכיל 3 אנשים) מבנץ לעוגי. את המסלול ניתן להדפיס כרשימה של שמות האנשים שבמסלול (השדה _name) עם חיצים בין השמות, בשני אופנים:

מימין לשמאל: Ugi<-Arik<-Bentz

משמאל לימין: Bentz->Arik->Ugi

(יכול להיות שיש מסלולים נוספים מבנץ לעוגי, ואולי אף מסלול קצר יותר, אם עוגי נמצא בעצמו במערך החברים של בנץ).

א. (14 נקודות)

עליכם לממש שירות זה למערכת - לממש את השיטה הלא-סטטית:

```
public void printPathRightToLeft(Person target, int k)
```

שיטה זו אמורה להיות מופעלת על מופע של המחלקה Person, ועליה למצוא ולהדפיס מסלול חברים כלשהו (לאו דווקא הקצר ביותר) באורך (מספר אנשים במסלול, כולל שני הקצוות) שלא עולה על k, מן האדם עליו הופעלה השיטה ועד לאדם היעד target. המסלול יודפס מימין לשמאל. למשל בדוגמה שניתנה לעיל, אם מפעילים את השיטה על האדם Bentz, כאשר פרמטר ה-target הוא הפניה (reference) לאדם Ugi, ופרמטר k=5, פלט אפשרי של השיטה יהיה:

Ugi<-Arik<-Bentz

אם לא קיים מסלול כנדרש, הפונקציה תדפיס הודעה:

No path found.

על המימוש להיות רקורסיבי.

ב. (3 נקודות)

מה סיבוכיות זמן הריצה של השיטה שכתבתם? מהי סיבוכיות הזיכרון (חשבו על עומק הרקורסיה)? הסבירו! השתמשו ב n ו/או ב k כדי לתאר את הסיבוכיות. תארו את סיבוכיות המקרה הגרוע ביותר.

המשך השאלה בעמוד הבא...

67.101
16/6/89

ג. (13 נקודות)

כעת נדרש לשכלל את השיטה, כך ש:

1. במהלך הרקורסיה לא יבוצעו זמן ריצה על בדיקת מסלולים עם מעגל (מסלול שבו אותו אדם – אותו אובייקט בזיכרון – מופיע כמה פעמים).
2. המסלול הנבחר יודפס הפעם **משמאל לימין**, כך שעבור אותם הפרמטרים כמו בדוגמה מסעיף א', יוכל להימצא אותו מסלול כמו קודם, רק הפעם הוא יודפס כך:

Bentz->Arik->Ugi

ממשו את השיטה (שוב, **לא-סטטית**):

```
public void printPathLeftToRight(Person target, int k)
```

שתבצע אותה משימה כמו בסעיף א' רק עם שני השינויים המפורטים לעיל. לצורך מימוש פונקציה זו אתם רשאים לייצר מערכים/אובייקטים בגודל שתלוי בקלט. הפתרון צריך להיות יעיל בזמן ובזיכרון עבור מקרים בהם k קטן משמעותית מ n (לדוגמה עבור רשת עם מליון אנשים, כשמחפשים מסלול חברים באורך 6).

ד. (3 נקודות)

מהי סיבוכיות זמן הריצה וסיבוכיות הזיכרון של השיטה שכתבתם בסעיף ג? הסבירו! השתמשו ב n ו/או ב k כדי לתאר את הסיבוכיות. תארו את סיבוכיות המקרה הגרוע ביותר.

67-101
16/6 8PS

שאלה 4 (33 נקודות)

חיפוש ומיון

בשאלה זו מותר להשתמש במבני נתונים דינמיים כמפורט להלן. הניחו כי קיימת שתי מחלקות המממשות רשימות באמצעות מערכים בעלי אורך דינמי (עם API המפורט להלן). כל השיטות במחלקות הנ"ל ממומשות עבורכם, והנכם רשאים לבצע בהם שימוש. הנח כי הרשימה מתחילה ממיקום 0. להלן הקטעים הרלוונטי מהגדרת כל אחת משתי המחלקות:

```
public class IntArrayList {
    public IntArrayList(); // Create an empty list, complexity O(1)

    // Append the provided element to the end of this list,
    // Complexity of appending n elements to the same list is O(n)
    public void append(int element);

    // Insert the provided element at the specified position in this list,
    // Complexity O(number of elements in the list)
    public void insert(int index, int element);

    // Return the element at the specified position in this list, O(1)
    public int get(int index);

    // Returns the number of elements in this list, O(1)
    public int size();
}

public class IndexEntryArrayList {
    // The functionality and the complexity of the methods here
    // is exactly the same as of the corresponding methods of IntArrayList

    public IndexEntryArrayList();

    public void append(IndexEntry element);

    public void insert(int index, IndexEntry element);

    public IndexEntry get(int index);

    public int size();
}
```

יזמית צעירה הקימה חברה המתמחה בחיפוש של מילים בקבצי מסמכים. במהדורה הראשונה של המוצר שם המחלקה המרכזית הוא DocumentNaive, והיא מייצגת מסמך על ידי מערך (array) של מילים, כאשר כל מילה מיוצגת על ידי מחרוזת (String). שימו לב שהמילה הראשונה במסמך תופיע בתא 0 של המערך.

```
public class DocumentNaive{
    private String[] _doc;

    public DocumentNaive(File file){
        // reads a document from a file, creating a String
        // for each word, and storing them in the array _doc
    }

    public IntArrayList search(String word){
        //To be completed by you...
    }
}
```

המשך השאלה בעמוד הבא...

67-101
 'e/f"8PA

א. (6 נקודות)

ממשו את השיטה search המקבלת מילה (String) ומחזירה רשימה של המיקומים של המילה במסמך (אובייקט מטיפוס IntArrayList). באם המילה אינה מופיעה במסמך, על השיטה להחזיר null.

לאחר שהלקוחות התלוננו על האיטיות של החיפוש, הוחלט לשפר את הביצועים על ידי הכנת מפתח (index) שישפר את זמן החיפוש.

```
public class DocumentImproved {
    private String[] _doc;
    private IndexEntryArrayList _index;
    public DocumentImproved(File file){
        _doc = ... // read an array of Strings from file
        buildIndex();
    }
    public IntArrayList search(String word){
        //To be completed by you...
    }
    private void buildIndex(){
        //To be completed by you...
    }
}
```

כאשר כל שורה במפתח מכילה מילה ורשימת כל מיקומיה במסמך כאובייקט מהמחלקה IndexEntry המוגדרת להלן:

```
public class IndexEntry{
    private String _word;
    private IntArrayList _locationList;

    public IndexEntry(String word){
        _word = word;
        _locationList = new IntArray();
    }

    public void addLocation(int location){
        _locationList.append(location);
    }

    public String getWord(){
        return _word;
    }

    public IntArrayList getList(){
        return _locationList;
    }
}
```

ב. (19 נקודות)

ממשו את השיטות search() ו-buildIndex() של המחלקה המשופרת (DocumentImproved).

המשך השאלה בעמוד הבא...

67 - 101
16 / 8 " 8 PA

ג. (4 נקודות)

חשבו את סיבוכיות זמן הריצה של שתי הפונקציות שמימשת בסעיף ב'. הסבירו בקצרה. ניתן להניח את ההנחות הבאות:

השוואה בין שתי מחרוזות מתבצעת בסיבוכיות $O(1)$
 n פעולות `append()` לאותה רשימה לוקחות בסה"כ זמן $O(n)$
פעולת `insert()` לוקחת (אורך הרשימה) O
פעולות `get()` ו-`size()` לוקחות $O(1)$

ד. (4 נקודות)

כעבור זמן, התברר שהלקוחות מעוניינים לחפש דוקא תתי-מחרוזות (המהוות חלק ממילה) ולא מילים שלמות. ניתן להפעיל את השיטה הבאה על מחרוזת מטיפוס `String` על מנת לבדוק האם המחרוזת `substr` מוכלת במחרוזת שעליה הופעלה השיטה:

```
public boolean matches(String substr);
```

האם העבודה שהושקעה בבניית המפתח בסעיף ב' (בהנחה שלא ניתן כעת לשנות את מימושן) ירדה לטמיון? הסבירו.

בהצלחה!!